

Concurrency Control

LAB 1:

step 1

The image shows two side-by-side PostgreSQL query editors. The left editor contains a SQL script for Step 1, and the right editor contains a SQL script for Step 2. Both editors have a toolbar with various icons for file operations, query execution, and settings. Below the query editors, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The left editor's 'Data Output' tab shows the result of the first query: 'UPDATE 1' and 'Query returned successfully in 267 msec.'. The right editor's 'Data Output' tab shows a message: 'No data output. Execute a query to get output.'

```
1 -- Step 1
2 BEGIN ;
3 UPDATE film SET rental_rate = 4.99
4 WHERE film_id = 1 ;
5 -- Holds X-Lock on film_id = 1
6
7 -- Step 3 (Run after Step 2 hangs)
8 COMMIT;
9 -- Releases the lock
```

```
1 -- Step 2
2 BEGIN ;
3 UPDATE film SET rental_rate = 5.99
4 WHERE film_id = 1 ;
5 --BLOCKS IMMEDIATELY !
6 -- Waits for Session A
7 -- Step 4 (Observing)
8 -- Executes automatically after A commits .
9 COMMIT;
10
```

UPDATE 1
Query returned successfully in 267 msec.

No data output. Execute a query to get output.

Total rows: Query complete 00:00:00.267

step 2

The image displays two side-by-side screenshots of a PostgreSQL client interface, likely pgAdmin, showing a sequence of steps for a database transaction. Both screenshots are for a connection named 'dvdrental/postgres@PostgreSQL 18'.

Left Screenshot (Step 1):

- Query Tab:** Shows a SQL script with the following steps:
 - Step 1
 - BEGIN ;
 - UPDATE film SET rental_rate = 4.99
 - WHERE film_id = 1 ;
 - Holds X-Lock on film_id = 1
 - Step 3 (Run after Step 2 hangs)
 - COMMIT;
 - Releases the lock
- Messages Tab:** Shows the message: 'UPDATE 1' and 'Query returned successfully in 267 msec.'
- Status Bar:** 'Total rows: Query complete 00:00:00.267'.

Right Screenshot (Step 2):

- Query Tab:** Shows a SQL script with the following steps:
 - Step 2
 - BEGIN ;
 - UPDATE film SET rental_rate = 5.99
 - WHERE film_id = 1 ;
 - BLOCKS IMMEDIATELY !
 - Waits for Session A
 - Step 4 (Observing)
 - Executes automatically after A commits .
 - COMMIT;
 -
- Data Output Tab:** Shows a message: 'Waiting for the query to complete...' with a circular arrow icon.
- Status Bar:** 'Total rows: Waiting for the query to complete... 00:00:03.565'.

step 3

The screenshot displays two side-by-side terminal windows running PostgreSQL commands via pgcli.

Left Window:

- Header: dvdrental/postgres@PostgreSQL 18
- Toolbar: Includes icons for file operations, filters, and a "No limit" dropdown.
- Query History: Shows a sequence of steps:
 - Step 1
 - BEGIN ;
 - UPDATE film SET rental_rate = 4.99
 - WHERE film_id = 1 ;
 - Holds X-Lock on film_id = 1
 - Step 3 (Run after Step 2 hangs)
 - COMMIT;
 - Releases the lock
- Data Output / Messages: Shows "COMMIT" and "Query returned successfully in 376 msec."
- Status Bar: "Total rows: Query complete 00:00:00.376"

Right Window:

- Header: dvdrental/postgres@PostgreSQL 18
- Toolbar: Similar to the left window, but includes additional navigation icons.
- Query History: Shows a sequence of steps:
 - Step 2
 - BEGIN ;
 - UPDATE film SET rental_rate = 5.99
 - WHERE film_id = 1 ;
 - BLOCKS IMMEDIATELY !
 - Waits for Session A
 - Step 4 (Observing)
 - Executes automatically after A commits .
 - COMMIT;
- Data Output / Messages: Shows "UPDATE 1" and "Query returned successfully in 19 secs 590 msec."
- Confirmation Boxes: Two green boxes at the bottom right state:
 - "✓ Query returned successfully in 19 secs 590 msec. ✕"
 - "✓ Query returned successfully in 376 msec. ✕"

step 4

The image displays two side-by-side terminal windows running pgcli, connected to a PostgreSQL 18 database. Both windows are titled 'dvdrental/postgres@PostgreSQL 18*'. The left window shows a transaction in progress, with a tooltip 'Execute script' and 'F5' visible. The SQL commands are: `Step 1`, `BEGIN ;`, `UPDATE film SET rental_rate = 4.99`, `WHERE film_id = 1 ;`, `-- Holds X-Lock on film_id = 1`, `-- Step 3 (Run after Step 2 hangs)`, `COMMIT;`, and `-- Releases the lock`. The 'Data Output' section shows 'COMMIT' and 'Query returned successfully in 376 msec.'. The right window shows the next step: `Step 2`, `BEGIN ;`, `UPDATE film SET rental_rate = 5.99`, `WHERE film_id = 1 ;`, `--BLOCKS IMMEDIATELY !`, `-- Waits for Session A`, `-- Step 4 (Observing)`, `-- Executes automatically after A commits .`, and `COMMIT;`. The 'Data Output' section shows 'COMMIT' and 'Query returned successfully in 252 msec.'. A green box at the bottom right of the right window highlights the success message: '✓ Query returned successfully in 252 msec. ✕'. The status bar at the bottom of both windows shows 'Total rows: Query complete 00:00:00.376' and 'Total rows: Query complete 00:00:00.252' respectively.

LAB 2 :

step 1

The screenshot displays a PostgreSQL client interface with two tabs, both titled 'dvdrental/postgres@PostgreSQL 18*'. The left tab shows a SQL query with three steps: Step 1 (BEGIN), Step 2 (SELECT rental_rate FROM film WHERE film_id = 10;), and Step 3 (SELECT rental_rate FROM film WHERE film_id = 10;). The query is executed, and the result is shown in the 'Data Output' pane as a single row with the value 4.99. The right tab shows a SQL query with two steps: Step 1 (BEGIN) and Step 2 (UPDATE film SET rental_rate = 0.99 WHERE film_id = 10;). The query is executed, and the result is shown in the 'Data Output' pane as a single row with the value 0.99. The interface includes a toolbar with various icons for file operations, a 'Query' pane, a 'Query History' pane, and a 'Data Output' pane. The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.513' for the left query, and 'Total rows: 1' and 'Query complete 00:00:00.185' for the right query.

```
--Step 1
BEGIN ;
SELECT rental_rate FROM film
WHERE film_id = 10;
-- Result : 4.99 (S-Lock released)
-- Step 3
SELECT rental_rate FROM film
WHERE film_id = 10;
--NEW RESULT : 0.99 !
COMMIT;
```

rental_rate
4.99

```
--Step 2
BEGIN ;
UPDATE film SET rental_rate = 0.99
WHERE film_id = 10;
COMMIT;
--Modifies data and commits
--while A is still active
```

rental_rate
0.99

COMMIT

Query returned successfully in 185 msec.

Total rows: 1 Query complete 00:00:00.513 CRLF Ln 4, Col 20

Total rows: 1 Query complete 00:00:00.185 CRLF Ln 4, Col 18

step 2

The screenshot displays two side-by-side instances of the pgAdmin web interface, both connected to a PostgreSQL database named 'dvddental'.

Left Interface:

- Query Editor:** Shows a multi-line SQL query:


```
--Step 1
BEGIN ;
SELECT rental_rate FROM film
WHERE film_id = 10;
-- Result : 4.99 (S-Lock released)
-- Step 3
SELECT rental_rate FROM film
WHERE film_id = 10;
--NEW RESULT : 0.9 9!
COMMIT;
```
- Data Output:** The 'Data Output' tab is active, displaying a single row from the 'film' table:

rental_rate
4.99

Right Interface:

- Query Editor:** Shows a different SQL query:


```
--Step 2
BEGIN ;
UPDATE film SET rental_rate = 0.99
WHERE film_id = 10;
COMMIT;
--Modifies data and commits
--while A i s s t i l l active
```
- Data Output:** The 'Messages' tab is active, showing the execution result:


```
COMMIT
Query returned successfully in 238 msec.
```

step 3

The image shows two side-by-side pgAdmin 4 windows connected to a PostgreSQL 18 database named 'dvdrental'.

Left Window:

- Query:**

```
1 --Step 1
2 BEGIN ;
3 SELECT rental_rate FROM film
4 WHERE film_id = 10;
5 -- Result : 4.99 (S-Lock released)
6 -- Step 3
7 SELECT rental_rate FROM film
8 WHERE film_id = 10;
9 --NEW RESULT : 0.99!
10 COMMIT;
11
```
- Data Output:** Shows a table with one row:

rental_rate
0.99
- Messages:** Empty.
- Notifications:** Empty.
- Status:** Total rows: 1, Query complete 00:00:00.238

Right Window:

- Query:**

```
1 --Step 2
2 BEGIN ;
3 UPDATE film SET rental_rate = 0.99
4 WHERE film_id = 10;
5 COMMIT;
6 --Modifies data and commits
7 --while A is still active
```
- Data Output:** Empty.
- Messages:** Shows the message: `COMMIT`
- Notifications:** Empty.
- Status:** Query returned successfully in 238 msec.

LAB 3:

step 1

Query	Query History	Query	Query History
1 --Step 1 2 BEGIN ; 3 UPDATE film SET rental_rate = 2.99 4 WHERE film_id = 100; 5 -- Step 3 6 UPDATE film SET rental_rate = 3.99 7 WHERE film_id = 200; 8 -- Blocks (Waiting for B) 9 COMMIT;		1 --Step 2 2 BEGIN ; 3 UPDATE film SET rental_rate = 2.99 4 WHERE film_id = 200; 5 -- Step 4 6 UPDATE film SET rental_rate = 3.99 7 WHERE film_id = 100; 8 -- BOOM! ERROR : deadlock detected	
Data Output	Messages	Data Output	Messages
UPDATE 1 Query returned successfully in 269 msec.		COMMIT Query returned successfully in 238 msec.	

step 2

The image shows two side-by-side PostgreSQL query editors, both connected to 'dvdrental/postgres@PostgreSQL 18'. Each editor has a toolbar with icons for file operations, query execution, and settings. Below the toolbar is a 'Query' tab with a line editor and a 'Query History' tab.

Left Editor Query:

```
1 --Step 1
2 BEGIN ;
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 100;
5 -- Step 3
6 UPDATE film SET rental_rate = 3.99
7 WHERE film_id = 200;
8 -- Blocks ( Waiting for B)
9 COMMIT;
```

Left Editor Messages:

```
UPDATE 1
Query returned successfully in 269 msec.
```

Right Editor Query:

```
1 --Step 2
2 BEGIN ;
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 200;
5 -- Step 4
6 UPDATE film SET rental_rate = 3.99
7 WHERE film_id = 100;
8 -- BOOM! ERROR : deadlock detected
```

Right Editor Messages:

```
UPDATE 1
Query returned successfully in 256 msec.
```

At the bottom of the right editor, a green status bar indicates: ✓ Query returned successfully in 256 msec. ✓

step 3

The image shows two side-by-side PostgreSQL query editors. Both are connected to 'dvdrental/postgres@PostgreSQL 18'.

Left Editor:

```
1 --Step 1
2 BEGIN ;
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 100;
5 -- Step 3
6 UPDATE film SET rental_rate = 3.99
7 WHERE film_id = 200;
8 -- Blocks ( Waiting for B)
9 COMMIT;
```

Right Editor:

```
1 --Step 2
2 BEGIN ;
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 200;
5 -- Step 4
6 UPDATE film SET rental_rate = 3.99
7 WHERE film_id = 100;
8 -- BOOM! ERROR : deadlock detected
```

Messages Panel (Left):

Total rows: 1
Waiting for the query to complete... 00:00:12.532

Messages Panel (Right):

UPDATE 1
Query returned successfully in 256 msec.

Status Bar (Left): CRLF Ln 7, Col 21

Status Bar (Right): Total rows: 1 Query complete 00:00:00.256 CRLF Ln 4, Col 21

step 4

The image shows two side-by-side PostgreSQL query editors, both connected to 'dvdrental/postgres@PostgreSQL 18'. The left editor shows a script with four steps. Step 4, 'UPDATE film SET rental_rate = 3.99 WHERE film_id = 200;', is highlighted in blue. The right editor shows the same script, but Step 4 is also highlighted in blue, and the final line is '-- BOOM! ERROR : deadlock detected'. Below the query editors are two 'Messages' panels. The left panel shows a successful update on 'UPDATE 1' with a completion time of 00:00:48.618. The right panel shows an 'ERROR: deadlock detected' message with details about processes 10608 and 2656 waiting for ShareLocks. A green box with a checkmark and the text 'Query returned successfully in 48 secs 618 msec.' is overlaid on the right message panel.

```
--Step 1
BEGIN ;
UPDATE film SET rental_rate = 2.99
WHERE film_id = 100;
-- Step 3
UPDATE film SET rental_rate = 3.99
WHERE film_id = 200;
-- Blocks ( Waiting for B)
COMMIT;
```

```
--Step 2
BEGIN ;
UPDATE film SET rental_rate = 2.99
WHERE film_id = 200;
-- Step 4
UPDATE film SET rental_rate = 3.99
WHERE film_id = 100;
-- BOOM! ERROR : deadlock detected
```

UPDATE 1
Query returned successfully in 48 secs 618 msec.

ERROR: deadlock detected
Process 10608 waits for ShareLock on transaction 986; blocked by process 2656.
Process 2656 waits for ShareLock on transaction 987; blocked by process 10608.

SQL state: 40P01
Detail: Process 10608 waits for ShareLock on transaction 986; blocked by process 2656.
Process 2656 waits for ShareLock on transaction 987; blocked by process 10608.
Hint: See server log for details of the lock contention.

✓ Query returned successfully in 48 secs 618 msec. ✕

Total rows:	Query complete 00:00:48.618	CRLF	Ln 7, Col 21
-------------	-----------------------------	------	--------------

Total rows:	Query complete 00:00:01.103	CRLF	Ln 7, Col 21
-------------	-----------------------------	------	--------------

LAB 4:

step 1

The image shows two side-by-side PostgreSQL query editors. Both are connected to 'dvdrental/postgres@PostgreSQL 18'.

Left Editor Query:

```
1  -- Step 1
2  BEGIN ;
3  SELECT *
4  FROM inventory
5  WHERE inventory_id = 10 FOR UPDATE ;
6  -- Locks the row proactively
7
8  -- Step 3
9  INSERT INTO rental
10 ( rental_date , inventory_id , customer_id , staff_id )
11 VALUES (NOW( ) , 10 , 1 , 1 ) ;
12 COMMIT ;
```

Right Editor Query:

```
1  -- Step 2
2  BEGIN ;
3  SELECT
4  *
5  FROM inventory
6  WHERE inventory_id = 10 FOR UPDATE ;
7  -- BLOCKS IMMEDIATELY
8  -- Step 4
9  -- Unblocks after A commits .
10 -- App logic detects i t 's already rented !
11 ROLLBACK ;
```

Data Output (Left Editor):

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2006-02-15 10:09:17

Total rows: 1 | Query complete 00:00:00.521

Right Editor Data Output:

❗ No data output. Execute a query to get output.

step 2

The image displays two side-by-side screenshots of a PostgreSQL client interface, likely pgAdmin, showing a deadlock scenario. Both windows are connected to 'dvdrental/postgres@PostgreSQL 18'.

Left Window (Successful Transaction):

- Query:**

```
1 -- Step 1
2 BEGIN ;
3 SELECT *
4 FROM inventory
5 WHERE inventory_id = 10 FOR UPDATE ;
6 -- Locks the row proactively
7
8 -- Step 3
9 INSERT INTO rental
10 ( rental_date , inventory_id , customer_id , staff_id )
11 VALUES (NOW( ) , 10 , 1 , 1 ) ;
12 COMMIT;
```
- Data Output Table:**

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2006-02-15 10:09:17
- Status Bar:** Total rows: 1 | Query complete 00:00:00.327 | CRLF | Ln 5, Col 37

Right Window (Blocked Transaction):

- Query:**

```
1 -- Step 2
2 BEGIN ;
3 SELECT *
4 FROM inventory
5 WHERE inventory_id = 10 FOR UPDATE ;
6 -- BLOCKS IMMEDIATELY
7 -- Step 4
8 -- Unblocks after A commits .
9 -- App logic detects i t 's already rented !
10 ROLLBACK ;
```
- Data Output Table:** (Empty)
- Status Bar:** Total rows: | Waiting for the query to complete... 00:00:05.813 | CRLF | Ln 5, Col 37

step 3

Two PostgreSQL query editors are shown side-by-side, both connected to 'dvdrental/postgres@PostgreSQL 18'. The left editor displays a query with the following steps:

```
1  -- Step 1
2  BEGIN ;
3  SELECT *
4  FROM inventory
5  WHERE inventory_id = 10 FOR UPDATE ;
6  -- Locks the row proactively
7
8  -- Step 3
9  INSERT INTO rental
10 (rental_date, inventory_id, customer_id, staff_id)
11 VALUES (NOW(), 10, 1, 1) ;
12 COMMIT;
```

The right editor displays a query with the following steps:

```
1  -- Step 2
2  BEGIN ;
3  SELECT *
4  FROM inventory
5  WHERE inventory_id = 10 FOR UPDATE ;
6  -- BLOCKS IMMEDIATELY
7  -- Step 4
8  -- Unblocks after A commits .
9  -- App logic detects it's already rented !
10 ROLLBACK ;
```

Below the queries, the 'Data Output' tab is active, showing a table with one row of data:

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
10	2	2	2006-02-15 10:09:17

Messages: COMMIT
Query returned successfully in 131 msec.

Total rows: Query complete 00:00:00.131

step 4

The image displays two side-by-side PostgreSQL query editors, both connected to 'dvdrental/postgres@PostgreSQL 18'. Each editor has a toolbar with icons for file operations, query execution, and settings. Below the toolbar are tabs for 'Query' and 'Query History'.

Left Editor (Steps 1-14):

```
1  -- Step 1
2  BEGIN ;
3  SELECT *
4  FROM inventory
5  WHERE inventory_id = 10 FOR UPDATE ;
6  -- Locks the row proactively
7
8  -- Step 3
9  INSERT INTO rental
10 (rental_date, inventory_id, customer_id, staff_id)
11 VALUES (NOW(), 10, 1, 1) ;
12 COMMIT;
```

Right Editor (Steps 1-11):

```
1  -- Step 2
2  BEGIN ;
3  SELECT *
4  FROM inventory
5  WHERE inventory_id = 10 FOR UPDATE ;
6  -- BLOCKS IMMEDIATELY
7  -- Step 4
8  -- Unblocks after A commits .
9  -- App logic detects it's already rented !
10 ROLLBACK ;
```

Data Output / Messages:

Left Editor: COMMIT
Query returned successfully in 131 msec.

Right Editor: ROLLBACK
Query returned successfully in 250 msec.

Status Bar: Total rows: Query complete 00:00:00.131 (Left) / Total rows: Query complete 00:00:00.250 (Right)

A green notification box at the bottom right of the right editor states: "✓ Query returned successfully in 250 msec. ✕"